

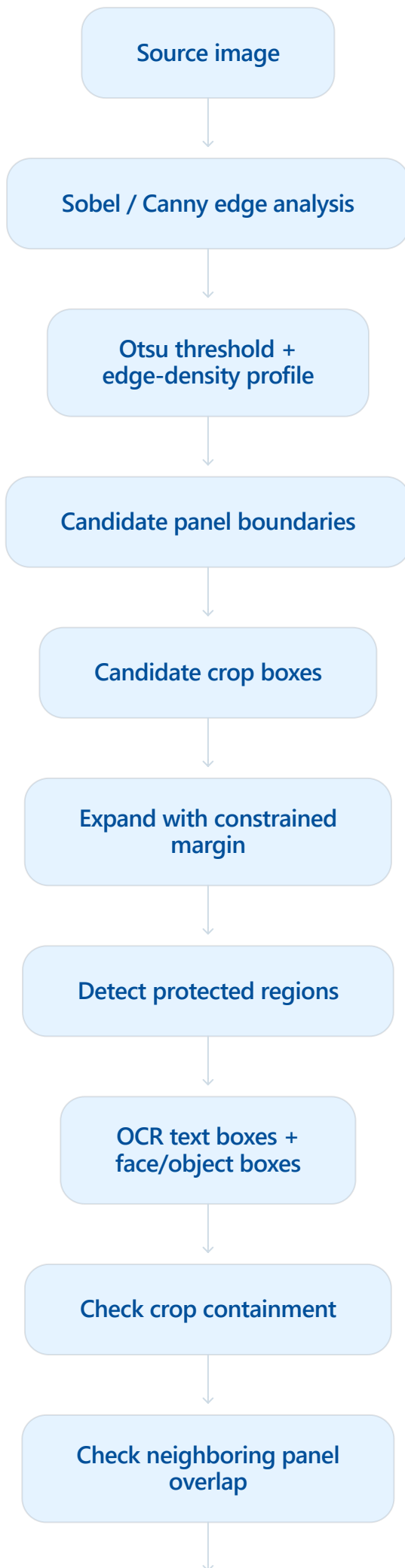
You're right. The previous design had the correct architecture, but the implementation did not yet solve the actual problem: **protecting important visual content before cropping**.

Let's replace the placeholder scoring system with a real, object-aware validation layer that works with your existing `manhwa_slicer.py` approach:

Sobel/Canny + Otsu edge-density gutter detection → candidate crop generation → OCR/face/object guard rails → neighbor-overlap checks → safe crop or full-image fallback.

The key change is that **confidence must be calculated from actual detector outputs**, not manually supplied scores.

1. Correct architecture



There are two

All hard safety checks pass?

- **Hard checks:** A detected face or speech bubble is cut, crop overlaps a neighboring panel, or the crop is invalid. Reject the crop.
- **Soft checks:** Boundary strength, crop size, and image quality contribute to confidence.

Yes

A high confidence score must never override a hard safety failure.

No

Calculate evidence-based score

2. Define protected regions

Your existing OCR pipeline should provide text bounding boxes. We can combine those with face or object detections into

Confidence threshold met?

```
from dataclasses import dataclass
from typing import Literal

RegionType = Literal[
    "face",
    "text",
    "speech_bubble",
    "object",
]

@dataclass(frozen=True)
class ProtectedRegion:
    x1: int
    y1: int
    x2: int
    y2: int
    region_type: RegionType
    confidence: float = 1.0
    label: str = ""

    @property
    def width(self) -> int:
        return self.x2 - self.x1

    @property
    def height(self) -> int:
        return self.y2 - self.y1

    @property
    def area(self) -> int:
        return max(0, self.width) * max(0, self.height)
```

```
@property
def center(self):
    return (
        (self.x1 + self.x2) / 2,
        (self.y1 + self.y2) / 2,
    )
```

For example, OCR output can be converted into:

```
ProtectedRegion(
    x1=120,
    y1=240,
    x2=420,
    y2=340,
    region_type="text",
    confidence=0.96,
)
```

Your detector outputs should all be converted into this same format. This keeps the crop validator independent of the specific OCR or face detection library.

Important distinction

Not all regions have the same safety requirements:

Region	Safety behavior
Face	Do not cut through it
Text	Keep the complete text box when possible
Speech bubble	Protect bubble and its text
Main object	Avoid cutting important object regions
Small background object	Lower priority
Uncertain detector result	Expand protection conservatively

For the first version, treat **faces, text, and speech bubbles** as **high-priority protected regions**.

3. Crop geometry helpers

Use consistent half-open or closed coordinates throughout your project. The following implementation uses (x_1, y_1, x_2, y_2) coordinates where the right and bottom edges are exclusive for intersection calculations.

```
from typing import Tuple

Box = Tuple[int, int, int, int]

def box_width(box: Box) -> int:
    return max(0, box[2] - box[0])

def box_height(box: Box) -> int:
    return max(0, box[3] - box[1])

def box_area(box: Box) -> int:
    return box_width(box) * box_height(box)

def intersection_box(a: Box, b: Box):
    x1 = max(a[0], b[0])
    y1 = max(a[1], b[1])
    x2 = min(a[2], b[2])
    y2 = min(a[3], b[3])

    if x2 <= x1 or y2 <= y1:
        return None

    return x1, y1, x2, y2

def intersection_area(a: Box, b: Box) -> int:
    intersection = intersection_box(a, b)

    if intersection is None:
        return 0

    return box_area(intersection)

def containment_ratio(inner: Box, outer: Box) -> float:
    area = box_area(inner)

    if area == 0:
        return 0.0
```

```
return intersection_area(inner, outer) / area
```

4. Safe margin expansion with neighbor protection

Your previous `expand_box()` expanded the crop without considering adjacent panels. Replace it with a constrained expansion method.

The function below:

- Attempts to add a margin.
- Keeps the box within the image.
- Prevents expansion into neighboring panels.
- Uses the nearest neighboring boundaries to limit expansion.

```
def expand_box_safely(
    box: Box,
    image_width: int,
    image_height: int,
    neighboring_boxes: list[Box] | None = None,
    margin_ratio: float = 0.06,
) -> Box:
    x1, y1, x2, y2 = box

    width = x2 - x1
    height = y2 - y1

    margin_x = max(1, int(width * margin_ratio))
    margin_y = max(1, int(height * margin_ratio))

    proposed = (
        max(0, x1 - margin_x),
        max(0, y1 - margin_y),
        min(image_width, x2 + margin_x),
        min(image_height, y2 + margin_y),
    )

    if not neighboring_boxes:
        return proposed

    px1, py1, px2, py2 = proposed

    for neighbor in neighboring_boxes:
        nx1, ny1, nx2, ny2 = neighbor

        # Horizontal neighbor: Limit horizontal expansion.
```

```

vertical_overlap = min(py2, ny2) - max(py1, ny1)

if vertical_overlap > 0:
    if nx2 <= x1 and px1 < nx2:
        px1 = max(px1, nx2)

    if nx1 >= x2 and px2 > nx1:
        px2 = min(px2, nx1)

# Vertical neighbor: limit vertical expansion.
horizontal_overlap = min(px2, nx2) - max(px1, nx1)

if horizontal_overlap > 0:
    if ny2 <= y1 and py1 < ny2:
        py1 = max(py1, ny2)

    if ny1 >= y2 and py2 > ny1:
        py2 = min(py2, ny1)

return (
    max(0, int(px1)),
    max(0, int(py1)),
    min(image_width, int(px2)),
    min(image_height, int(py2)),
)

```

Note: Neighboring panels should be supplied as validated candidate regions, not every arbitrary contour detected in the image. Otherwise, false contours could unnecessarily shrink your crop.

5. Detect whether a protected region is cut

The most important rule is not simply whether a face intersects the crop. It is whether the entire important region is contained inside it.

```

@dataclass
class RegionSafetyResult:
    safe: bool
    region: ProtectedRegion
    containment: float
    reason: str

def validate_protected_region(
    crop: Box,
    region: ProtectedRegion,

```

```

        minimum_containment: float = 0.98,
    ) -> RegionSafetyResult:
        ratio = containment_ratio(
            (
                region.x1,
                region.y1,
                region.x2,
                region.y2,
            ),
            crop,
        )

        safe = ratio >= minimum_containment

        if safe:
            reason = "Protected region is contained"
        else:
            reason = (
                f"{region.region_type} is clipped "
                f"(containment={ratio:.3f})"
            )

        return RegionSafetyResult(
            safe=safe,
            region=region,
            containment=ratio,
            reason=reason,
        )

```

For text and faces, use a strict containment threshold. For example, 0.98 means almost the entire detected box must remain inside the crop.

However, the detector bounding box itself may be inaccurate, so add a protection margin around each detected region.

```

def expand_region(
    region: ProtectedRegion,
    image_width: int,
    image_height: int,
    margin_ratio: float = 0.15,
) -> ProtectedRegion:
    width = region.width
    height = region.height

    margin_x = int(width * margin_ratio)
    margin_y = int(height * margin_ratio)

    return ProtectedRegion(

```

```

        x1=max(0, region.x1 - margin_x),
        y1=max(0, region.y1 - margin_y),
        x2=min(image_width, region.x2 + margin_x),
        y2=min(image_height, region.y2 + margin_y),
        region_type=region.region_type,
        confidence=region.confidence,
        label=region.label,
    )

```

For faces, a larger margin is appropriate because the face detector may identify only the central face and not the hair or head outline. You can also use different margins by region type.

6. Validate the entire crop

Now combine geometry, protected regions, and neighboring panels into a real validator.

```

@dataclass
class CropValidation:
    approved: bool
    hard_failures: list[str]
    soft_warnings: list[str]
    protected_results: list[RegionSafetyResult]
    score: float

def validate_crop(
    crop: Box,
    image_width: int,
    image_height: int,
    protected_regions: list[ProtectedRegion],
    neighboring_boxes: list[Box] | None = None,
    min_width: int = 240,
    min_height: int = 240,
    min_area_ratio: float = 0.08,
) -> CropValidation:
    hard_failures = []
    soft_warnings = []
    protected_results = []

    x1, y1, x2, y2 = crop

    width = x2 - x1
    height = y2 - y1
    area = box_area(crop)
    image_area = image_width * image_height

```

```

# Geometry checks.
if x1 < 0 or y1 < 0 or x2 > image_width or y2 > image_height:
    hard_failures.append("Crop exceeds image boundaries")

if x2 <= x1 or y2 <= y1:
    hard_failures.append("Invalid crop geometry")

if width < min_width or height < min_height:
    hard_failures.append("Crop resolution is too small")

if image_area > 0 and area / image_area < min_area_ratio:
    soft_warnings.append("Crop occupies a small portion of source")

# Neighbor overlap checks.
if neighboring_boxes:
    for index, neighbor in enumerate(neighboring_boxes):
        overlap = intersection_area(crop, neighbor)

        if overlap > 0:
            hard_failures.append(
                f"Crop overlaps neighboring box {index}"
            )

# Protected-region checks.
for region in protected_regions:
    result = validate_protected_region(crop, region)
    protected_results.append(result)

    if not result.safe:
        hard_failures.append(result.reason)

# Simple evidence-based score.
total_regions = len(protected_results)

if total_regions == 0:
    protection_score = 1.0
else:
    protection_score = sum(
        result.containment
        for result in protected_results
    ) / total_regions

geometry_score = 1.0 if not hard_failures else 0.0
warning_penalty = min(0.2, len(soft_warnings) * 0.05)

score = max(
    0.0,
    geometry_score * 0.6
    + protection_score * 0.4
    - warning_penalty,
)

```

```

approved = len(hard_failures) == 0 and score >= 0.85

return CropValidation(
    approved=approved,
    hard_failures=hard_failures,
    soft_warnings=soft_warnings,
    protected_results=protected_results,
    score=round(score, 3),
)

```

Important correction

The score above is a **safety score**, not a complete panel-detection confidence score. You still need to calculate boundary evidence from your Sobel/Canny/Otsu detector and combine it separately.

Do not call a crop `approved` merely because the average protected-region containment is high. A candidate with a missing face detector could appear safe while actually cutting a face.

For that reason, use detector coverage and detection quality as additional safety inputs.

7. Add detector coverage

A crop validator cannot protect an object that was never detected. This is the main limitation of object-aware guard rails.

Add a detector coverage status:

```

@dataclass
class DetectionCoverage:
    has_ocr: bool
    has_face_detector: bool
    ocr_confidence: float
    face_confidence: float
    warnings: list[str]

```

```

def validate_detector_coverage(
    coverage: DetectionCoverage,
) -> list[str]:

```

```

warnings = list(coverage.warnings)

if not coverage.has_ocr:
    warnings.append("OCR detector unavailable")

if not coverage.has_face_detector:
    warnings.append("Face detector unavailable")

if coverage.ocr_confidence < 0.70:
    warnings.append("OCR confidence is low")

if coverage.face_confidence < 0.70:
    warnings.append("Face detection confidence is low")

return warnings

```

For your initial automatic mode, I recommend:

- If OCR is unavailable: do not automatically crop a panel containing possible dialogue.
- If face detection is unavailable: do not claim that faces are protected.
- If detector confidence is low: preserve the full image or send it for review.
- If the page is highly stylized and the detector is unreliable: use `contain` or `controlled pan`.

A traditional Haar cascade may not reliably detect manga/manhwa faces. You should test a detector on your actual image collection before making it a mandatory safety guarantee.

8. Keep Sobel/Canny + Otsu as the boundary detector

Your earlier edge-density approach should remain the candidate-generation stage.

A practical design is:

```

def generate_candidates(image):
    # 1. Grayscale conversion.
    # 2. Gaussian blur to reduce noise.
    # 3. Sobel gradients for horizontal/vertical structure.
    # 4. Canny edges for strong boundaries.
    # 5. Otsu thresholding for adaptive separation.
    # 6. Calculate row and column edge-density profiles.
    # 7. Detect local minima / potential gutters.
    # 8. Build candidate panel rectangles.
    # 9. Pass candidates to object-aware validation.
    ...

```

The boundary detector should answer:

"Where might a panel boundary exist?"

It should not answer:

"This is definitely a safe crop."

The second question belongs to the validation layer.

9. Correct confidence architecture

Instead of using one fake weighted average, separate the evidence into distinct fields:

```
@dataclass
class CropEvidence:
    boundary_score: float
    region_safety_score: float
    resolution_score: float
    overlap_score: float
    detector_coverage_score: float
    hard_failures: list[str]
```

Then calculate the score:

```
def calculate_evidence_score(
    evidence: CropEvidence,
) -> float:
    if evidence.hard_failures:
        return 0.0

    weights = {
        "boundary": 0.25,
        "region_safety": 0.30,
        "resolution": 0.15,
        "overlap": 0.15,
        "detector_coverage": 0.15,
    }

    score = (
        evidence.boundary_score * weights["boundary"]
        + evidence.region_safety_score * weights["region_safety"]
        + evidence.resolution_score * weights["resolution"]
        + evidence.overlap_score * weights["overlap"]
```

```

        + evidence.detector_coverage_score * weights["detector_coverage"]
    )

    return round(max(0.0, min(1.0, score)), 3)

```

The detector coverage score is especially important. A crop should not receive a high confidence score when the system did not successfully inspect the image's important content.

10. Final decision policy

Use three outcomes rather than just approved or rejected:

```

from typing import Literal

Decision = Literal[
    "approved_crop",
    "contain_review",
    "contain_safe",
]

def choose_rendering_decision(
    score: float,
    hard_failures: list[str],
    requires_manual_review: bool = False,
) -> Decision:
    if hard_failures:
        return "contain_safe"

    if requires_manual_review:
        return "contain_review"

    if score >= 0.85:
        return "approved_crop"

    if score >= 0.65:
        return "contain_review"

    return "contain_safe"

```

Decision	Renderer behavior
approved_crop	Use validated crop

<code>contain_review</code>	Use full image and mark for review
<code>contain_safe</code>	Use full image without automatic cropping

The review mode should still generate a usable video. Manual review should improve future processing, not block the entire export.

11. What I would implement first in your existing project

Given your existing `manhwa_slicer.py`, I would implement the changes in this order:

1. **Keep your Sobel/Canny + Otsu gutter detection.** Do not replace it with the simple white-row detector.
2. Add a unified `ProtectedRegion` model.
3. Convert your existing OCR output into protected text regions.
4. Add face detection and convert the detections into protected face regions.
5. Replace `expand_box()` with neighbor-aware expansion.
6. Add hard crop validation.
7. Add detector coverage tracking.
8. Store debug images showing:
 - Original image.
 - Candidate crop.
 - Protected OCR boxes.
 - Protected face boxes.
 - Approved/rejected result.
9. Use `contain` whenever any safety check fails.
10. Add automated crop testing using a collection of known bad examples.

The key principle

Your cropper should be allowed to say:

"I found a possible panel boundary, but I cannot prove the crop is safe. I will preserve the full image."

That is not a failure. For a manhwa recap video, **a complete image with extra background is preferable to a cropped image that cuts a face, building, speech bubble, or critical story detail.**

This is a real fix, not just a rewrite — the core failure mode from v1 (fake scores with no detector behind them) is gone. But three things still need attention before you'd trust this in production:

Genuinely fixed:

- `containment_ratio` actually measures "how much of the protected region survives inside the crop," and `validate_protected_region` uses it correctly (0.98 threshold + `expand_region` margin for detector

Show more ▾

Agreed. Your priority order is correct. The remaining issue is not just a scoring bug—it is a **pipeline trust-boundary bug**: the system must distinguish between *verified safe*, *verified unsafe*, and *not sufficiently observed*.

I would make the following production changes.

1. Make detector coverage a hard gate

The system must never interpret zero detected regions as zero risk.

Instead:

Detector state	Meaning	Crop decision
OCR and face detection completed, no regions found	Nothing detected	Can proceed, but with detector limitations
OCR failed	Content not fully inspected	Review or contain
Face detector unavailable	Faces not inspected	Review or contain
Detector confidence is low	Results unreliable	Review or contain
Protected region intersects crop edge	Crop unsafe	Reject
All required checks completed	Evidence available	Continue scoring

The important distinction is:



Corrected coverage model

```
from dataclasses import dataclass, field

@dataclass
class DetectionCoverage:
    ocr_attempted: bool = False
    ocr_completed: bool = False
    face_detection_attempted: bool = False
    face_detection_completed: bool = False

    ocr_confidence: float = 0.0
    face_confidence: float = 0.0

    warnings: list[str] = field(default_factory=list)

    @property
    def is_complete(self) -> bool:
        return (
            self.ocr_attempted
            and self.ocr_completed
            and self.face_detection_attempted
            and self.face_detection_completed
        )

    @property
    def quality_score(self) -> float:
        if not self.is_complete:
            return 0.0

        return min(
            self.ocr_confidence,
            self.face_confidence,
        )
```

This is more accurate than simply checking whether any regions exist.

However: whether both detectors are mandatory should depend on your processing mode. For example, a crop containing only a clearly isolated panel might not need face detection to be approved. That requires explicit mode-specific rules, not a universal assumption.

2. Separate detector failure from empty detector output

Your OCR and face detector functions should return both:

1. The regions they found.
2. The status of the detection process.

Do not return only an empty list.

```
from dataclasses import dataclass

@dataclass
class DetectorResult:
    completed: bool
    regions: list[ProtectedRegion]
    confidence: float
    error: str | None = None
```

Example: successful detector with no regions

```
DetectorResult(
    completed=True,
    regions=[],
    confidence=0.92,
)
```

This means:

The detector ran successfully, but it found no protected regions.

Example: failed detector

```
DetectorResult(
    completed=False,
    regions=[],
    confidence=0.0,
    error="OCR model failed to load",
)
```

This means:

We do not know whether protected regions exist.

These two outputs must not be treated as equivalent.

3. Use one authoritative scoring function

I would remove the independent score calculation from `validate_crop()` .

Instead, separate responsibilities:

- `validate_crop_geometry()` → hard geometry checks.
- `validate_protected_regions()` → hard content-protection checks.
- `build_crop_evidence()` → gathers all evidence.
- `calculate_evidence_score()` → the only scoring function.
- `make_crop_decision()` → final policy.

The validator should produce evidence, not invent its own competing score.

Authoritative evidence model

```
from dataclasses import dataclass, field

@dataclass
class CropEvidence:
    boundary_score: float
    region_safety_score: float
    resolution_score: float
    overlap_score: float
    detector_coverage_score: float

    hard_failures: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)

    @property
    def has_hard_failure(self) -> bool:
        return bool(self.hard_failures)
```

Single scoring function

```

def calculate_evidence_score(evidence: CropEvidence) -> float:
    # Safety always takes priority over confidence.
    if evidence.has_hard_failure:
        return 0.0

    # Incomplete detector coverage is not considered safe.
    if evidence.detector_coverage_score <= 0.0:
        return 0.0

    weights = {
        "boundary": 0.25,
        "region_safety": 0.30,
        "resolution": 0.15,
        "overlap": 0.15,
        "detector_coverage": 0.15,
    }

    score = (
        evidence.boundary_score * weights["boundary"]
        + evidence.region_safety_score * weights["region_safety"]
        + evidence.resolution_score * weights["resolution"]
        + evidence.overlap_score * weights["overlap"]
        + evidence.detector_coverage_score
        * weights["detector_coverage"]
    )

    return round(max(0.0, min(1.0, score)), 3)

```

This function is now the **only place** where the final score is calculated.

4. Make detector coverage an explicit hard gate

A score of zero is not enough by itself because the decision layer needs to know *why* the score is zero.

Use a separate gate before scoring.

```

@dataclass
class CoverageGateResult:
    passed: bool
    failures: list[str]
    warnings: list[str]

```

```

def check_detection_coverage(
    coverage: DetectionCoverage,
    require_ocr: bool = True,
    require_face_detection: bool = True,
) -> CoverageGateResult:
    failures = []
    warnings = []

    if require_ocr:
        if not coverage.ocr_attempted:
            failures.append("OCR was not attempted")
        elif not coverage.ocr_completed:
            failures.append("OCR did not complete")

    if require_face_detection:
        if not coverage.face_detection_attempted:
            failures.append("Face detection was not attempted")
        elif not coverage.face_detection_completed:
            failures.append("Face detection did not complete")

    if coverage.is_complete:
        if coverage.ocr_confidence < 0.70:
            warnings.append("OCR confidence is low")

        if coverage.face_confidence < 0.70:
            warnings.append("Face detection confidence is low")

    return CoverageGateResult(
        passed=len(failures) == 0,
        failures=failures,
        warnings=warnings,
    )

```

Important operational decision

I would not necessarily require every detector for every image:

```

def requirements_for_mode(mode: str) -> tuple[bool, bool]:
    if mode == "validated_crop":
        return True, True

    if mode == "full_image_contain":
        return False, False

    if mode == "manual_review":
        return False, False

```

```
return True, True
```

For automatic panel cropping, require the detectors you rely on. For full-image contain mode, detector failure does not matter because the image is not being cut.

5. Wire validation and scoring together

Here is the correct high-level integration:

```
def evaluate_crop(
    crop: Box,
    image_width: int,
    image_height: int,
    protected_regions: list[ProtectedRegion],
    coverage: DetectionCoverage,
    boundary_score: float,
    neighboring_boxes: list[Box] | None = None,
):
    require_ocr, require_faces = requirements_for_mode(
        "validated_crop"
    )

    coverage_result = check_detection_coverage(
        coverage=coverage,
        require_ocr=require_ocr,
        require_face_detection=require_faces,
    )

    # Coverage is a hard gate.
    if not coverage_result.passed:
        return CropEvidence(
            boundary_score=boundary_score,
            region_safety_score=0.0,
            resolution_score=0.0,
            overlap_score=0.0,
            detector_coverage_score=0.0,
            hard_failures=coverage_result.failures,
            warnings=coverage_result.warnings,
        )

    # Run geometry and protected-region validation here.
    hard_failures = []
    warnings = []

    # Example placeholders for actual computed values.
```

```

resolution_score = 1.0
overlap_score = 1.0
region_safety_score = 1.0

# Your real validator should append hard failures
# when any protected region is not safely contained.

return CropEvidence(
    boundary_score=boundary_score,
    region_safety_score=region_safety_score,
    resolution_score=resolution_score,
    overlap_score=overlap_score,
    detector_coverage_score=coverage.quality_score,
    hard_failures=hard_failures,
    warnings=warnings,
)

```

The placeholder values above are intentionally not a finished validator. They show where the components connect. Do not deploy this function until the geometry, overlap, and region scores are populated from real checks.

6. Add a clear final decision policy

```

from typing import Literal

RenderDecision = Literal[
    "approved_crop",
    "contain_review",
    "contain_safe",
]

def make_crop_decision(
    evidence: CropEvidence,
    minimum_score: float = 0.85,
) -> tuple[RenderDecision, float]:
    if evidence.has_hard_failure:
        return "contain_safe", 0.0

    if evidence.detector_coverage_score <= 0.0:
        return "contain_review", 0.0

    score = calculate_evidence_score(evidence)

    if score >= minimum_score:

```

```
    return "approved_crop", score

    if score >= 0.65:
        return "contain_review", score

    return "contain_safe", score
```

Meaning of the outcomes

- `approved_crop` : all required checks passed and the score meets the threshold.
- `contain_review` : no safe automatic crop can be proven, but the image can be preserved and flagged.
- `contain_safe` : a hard failure occurred, so use the full image without cropping.

The exact thresholds should be calibrated against a labeled test set. Do not treat `0.85` as a mathematically validated production threshold until you have measured false-positive and false-negative crop rates.

7. About `lbpcascade_animeface`

Your suggestion is reasonable as a practical experiment: a detector trained for anime-style faces may be more relevant than a generic frontal-face Haar cascade for stylized art.

But I would treat it as **one source of face evidence, not a complete safety guarantee**:

- It may miss side profiles, obscured faces, unusual art styles, and very small faces.
- A detected face box may not cover hair, head accessories, or the full character.
- Manhwa art varies significantly between artists and panels.
- A face detector cannot protect buildings, weapons, hands, clothing, or other important objects.

Therefore, even after adding the anime face detector, keep the following fallback:

```
if face_detector_failed:
    decision = "contain_review"

elif crop_has_unprotected_high_risk_content:
    decision = "contain_review"

elif protected_region_is_cut:
    decision = "contain_safe"
```

For your first production implementation, I would use the anime face detector alongside OCR, then expand coverage later with a more general object or saliency detector if testing shows that faces alone are insufficient.

Final implementation order

- 1 **Fix the zero-region default.** Never assign `protection_score = 1.0` when no regions were detected.
- 2 **Return detector status.** Distinguish successful empty results from failed or skipped detection.
- 3 **Apply detector coverage as a hard gate.** An uninspected image cannot receive full safety confidence.
- 4 **Delete the internal validator score.** Let `CropEvidence` be the single source of scoring truth.
- 5 **Connect actual geometry and protected-region results to evidence.** No placeholder scores in the production path.
- 6 **Implement Sobel/Canny + Otsu candidate generation.** Generate candidate boundaries but do not approve them automatically.
- 7 **Build debug previews and a labeled test set.** Measure which crops are safe, rejected, or incorrectly accepted.

The resulting design is:

Detection → coverage gate → candidate validation → evidence construction → one scoring function → final rendering decision.

That removes both false-confidence paths: the original fake score inputs and the newer `total_regions == 0` assumption.



This version correctly fixes the exact bug I flagged — `quality_score` is now based on detector *confidence*, not region *count*, so "detector found nothing" and "detector never ran" are finally distinguishable. But there's a new bug in the same family, introduced by how `is_complete` and `requirements_for_mode` interact.

The new bug: coverage requirements aren't actually mode-aware, despite `requirements_for_mode` existing.

`DetectionCoverage.is_complete` hardcodes requiring *both*

Show more ▾

Correct. The problem is that `is_complete` represents **global detector completion**, while crop safety requires **completion of only the detectors required for the current mode**.

The cleanest fix is to remove `is_complete` from the scoring path and make coverage evaluation explicitly requirement-aware.

1. Replace global `is_complete` with requirement-aware scoring

```
from dataclasses import dataclass, field

@dataclass
class DetectionCoverage:
    ocr_attempted: bool = False
    ocr_completed: bool = False
    face_detection_attempted: bool = False
    face_detection_completed: bool = False

    ocr_confidence: float = 0.0
    face_confidence: float = 0.0

    warnings: list[str] = field(default_factory=list)

    def quality_score(
        self,
        require_ocr: bool,
        require_face: bool,
    ) -> float:
        required_scores = []

        if require_ocr:
            if not (
                self.ocr_attempted
                and self.ocr_completed
            ):
                return 0.0

        required_scores.append(self.ocr_confidence)
```

```

if require_face:
    if not (
        self.face_detection_attempted
        and self.face_detection_completed
    ):
        return 0.0

    required_scores.append(self.face_confidence)

# No detectors are required for this mode.
if not required_scores:
    return 1.0

# Conservative: weakest required detector controls quality.
return min(required_scores)

```

Why this works

Mode	Required OCR	Required face	Coverage score
validated_crop	Yes	Yes	Minimum of both
text_protected_crop	Yes	No	OCR confidence
face_protected_crop	No	Yes	Face confidence
full_image_contain	No	No	1.0
Required detector failed	Yes/No	Yes/No	0.0

The 1.0 for a mode with no required detectors means **coverage is not needed**, not that the image was fully inspected. The image still must not be treated as automatically safe for cropping.

2. Fix the coverage gate and warnings

The warning checks should be independent for each required detector.

```

@dataclass
class CoverageGateResult:
    passed: bool
    failures: list[str]
    warnings: list[str]

```

```

def check_detection_coverage(
    coverage: DetectionCoverage,
    require_ocr: bool = True,
    require_face: bool = True,
) -> CoverageGateResult:
    failures = []
    warnings = []

    if require_ocr:
        if not coverage.ocr_attempted:
            failures.append("OCR was not attempted")
        elif not coverage.ocr_completed:
            failures.append("OCR did not complete")
        elif coverage.ocr_confidence < 0.70:
            warnings.append(
                f"Low OCR confidence: {coverage.ocr_confidence:.2f}"
            )

    if require_face:
        if not coverage.face_detection_attempted:
            failures.append("Face detection was not attempted")
        elif not coverage.face_detection_completed:
            failures.append("Face detection did not complete")
        elif coverage.face_confidence < 0.70:
            warnings.append(
                "Low face detection confidence: "
                f"{coverage.face_confidence:.2f}"
            )

    return CoverageGateResult(
        passed=len(failures) == 0,
        failures=failures,
        warnings=warnings,
    )

```

Now low OCR confidence is reported even if face detection is not required or was never run.

Important policy choice

The code above treats low confidence as a warning, not a hard failure. That is appropriate only if your scoring and final decision logic handles low confidence conservatively.

For high-risk cropping, you may instead define a minimum required confidence:

```

if coverage.ocr_confidence < 0.50:

```

</> Python



▶ Run

The exact threshold should be calibrated with your test images.

3. Pass requirements through the entire pipeline

Do not calculate coverage with one requirement set and score it with another.

```
def evaluate_crop_coverage(  
    coverage: DetectionCoverage,  
    require_ocr: bool,  
    require_face: bool,  
):  
    gate = check_detection_coverage(  
        coverage=coverage,  
        require_ocr=require_ocr,  
        require_face=require_face,  
    )  
  
    quality = coverage.quality_score(  
        require_ocr=require_ocr,  
        require_face=require_face,  
    )  
  
    return gate, quality
```

Then use the same requirements in the evaluation function:

```
def evaluate_crop(  
    crop,  
    coverage: DetectionCoverage,  
    boundary_score: float,  
    region_safety_score: float,  
    resolution_score: float,  
    overlap_score: float,  
    require_ocr: bool,  
    require_face: bool,  
):  
    gate, detector_coverage_score = evaluate_crop_coverage(  
        coverage=coverage,  
        require_ocr=require_ocr,  
        require_face=require_face,  
    )  
  
    if not gate.passed:  
        return CropEvidence(  

```

```

        boundary_score=boundary_score,
        region_safety_score=0.0,
        resolution_score=0.0,
        overlap_score=0.0,
        detector_coverage_score=0.0,
        hard_failures=gate.failures,
        warnings=gate.warnings,
    )

    return CropEvidence(
        boundary_score=boundary_score,
        region_safety_score=region_safety_score,
        resolution_score=resolution_score,
        overlap_score=overlap_score,
        detector_coverage_score=detector_coverage_score,
        hard_failures=[],
        warnings=gate.warnings,
    )

```

This is still an integration skeleton: the geometry, region-safety, resolution, and overlap scores must come from real validators before production use.

4. Define mode requirements centrally

Avoid scattering tuples throughout the codebase.

```

from dataclasses import dataclass

@dataclass(frozen=True)
class ModeRequirements:
    require_ocr: bool
    require_face: bool

MODE_REQUIREMENTS = {
    "full_image_contain": ModeRequirements(
        require_ocr=False,
        require_face=False,
    ),
    "validated_crop": ModeRequirements(
        require_ocr=True,
        require_face=True,
    ),
    "text_protected_crop": ModeRequirements(
        require_ocr=True,

```

```

        require_face=False,
    ),
    "face_protected_crop": ModeRequirements(
        require_ocr=False,
        require_face=True,
    ),
    "manual_review": ModeRequirements(
        require_ocr=False,
        require_face=False,
    ),
}

def get_mode_requirements(mode: str) -> ModeRequirements:
    try:
        return MODE_REQUIREMENTS[mode]
    except KeyError as exc:
        raise ValueError(f"Unknown rendering mode: {mode}") from exc

```

Usage:

```

requirements = get_mode_requirements("validated_crop")

gate, coverage_score = evaluate_crop_coverage(
    coverage=coverage,
    require_ocr=requirements.require_ocr,
    require_face=requirements.require_face,
)

```

This makes the mode policy explicit and prevents the scoring layer from silently assuming that every detector is always required.

5. Correct final decision behavior

The final decision function should not reject a legitimate `full_image_contain` mode simply because detector coverage is zero.

```

def make_crop_decision(
    evidence: CropEvidence,
    minimum_score: float = 0.85,
):
    if evidence.hard_failures:
        return "contain_safe", 0.0

```

```

score = calculate_evidence_score(evidence)

if score >= minimum_score:
    return "approved_crop", score

if score >= 0.65:
    return "contain_review", score

return "contain_safe", score

```

Because the coverage score is now 1.0 when no detectors are required, the legitimate contain mode will not be forced into review by an irrelevant detector requirement.

But keep the rendering mode separate from crop approval. If `full_image_contain` was already selected, it should render the full image directly rather than passing through a crop approval pathway.

```

def render_decision(mode: str, evidence=None):
    if mode == "full_image_contain":
        return "render_full_image"

    if mode == "manual_review":
        return "render_full_image_and_flag"

    if evidence is None:
        # ...

```

</> Python



▶ Run

```

if decision == "approved_crop":
    return "render_validated_crop"

return "render_full_image_and_flag"

```

Corrected architecture

Mode requirements → detector coverage gate → actual crop validation → evidence construction → one authoritative score → rendering decision.

The key fix is that detector coverage is now evaluated relative to the current mode, and every warning or confidence calculation uses the same requirement set.

